

CodeSentinel Scan Report -

<https://insurezeal.com>

Summary

Overall Risk: CRITICAL

The deployed website `insurezeal.com` – an Insurance Marketing Firm (IMF) application – exposes multiple severe security issues: a Supabase backend with publicly accessible plaintext credentials, unauthenticated API endpoints leaking business and personal data, and hardcoded user accounts embedded directly in the client-side JavaScript bundle. No authentication is required to access any of the discovered API surfaces.

Target Overview

Property	Value
Target	https://insurezeal.com
Type	Insurance Marketing Firm (IMF) dashboard application
Framework	React SPA (Vite build, single JS bundle ~1.6MB)
Backend	Node.js/Express at https://insurezeal.com/api
Database	Supabase (PostgreSQL) at https://ojpwampkbzjfupucjcmq.supabase.co
Internal Service	http://localhost:3001 (Claude AI extraction, policy save)
Business	Insure Zeal IMF Private Limited – IRDAI-registered insurance distributor

Observed Facts

1. Supabase Credentials Exposed in JS Bundle

The Supabase anon key and instance URL are hardcoded in the client-side JavaScript at line 36574 of the beautified bundle:

- **Supabase URL:** `https://ojpwampkbzjfupucjcmq.supabase.co`
- **Anon JWT Key:**
`eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJzdXBhYmFzZSIsInJlZiI6Im9qcHdhbXBBrYnpqZnVwdWNqY2IwIiwicm9sZSI6ImFub24iLCJpYXQiOiE3Nzg5MTAxODgsImV4cCI6MjA5NDQ4NjE4OH0.PKt5G8qd_qahVY_iIq_IDlrD62cCkdYJHnFwIvojsCI`

The JWT payload confirms: `"role": "anon"` , `"iss": "supabase"` , expiry `2094486188` (year 2036).

2. Hardcoded User Accounts in Client-Side JS

Ten user accounts are defined in a `PR` array (lines 7456–7526) with `username` , `password` , `role` , `name` , and `email` . Nine of them have the literal password `"string"` . One account has a real password:

Username	Password	Role	Email
gc1119	GS@2024	agent	gc@gmail.com
gc111	string	super_admin	admin@company.com
gc112	string	admin	admin@company.com
gc113	string	agent	agent@company.com
IZ054	string	agent	ashokbajaj@insurezeal.com
mk	string	agent	govind.insurezeal@gmail.com
IZ001	string	agent	insurezeal@gmail.com
gc114	string	ops_team	insurezeal@gmail.com
IZ136	string	agent	amit.insurezeal@gmail.com
as111	string	ops_team	govindchy11@gmail.com

Authentication is performed client-side via `TR.users.find(c => c.username === s && c.password === 1)` with no server-side validation.

3. API Base URLs Discovered

- `https://insurezeal.com/api` — primary backend
- `http://localhost:3001` — internal AI/policy service (not publicly accessible)

- <https://ojpwampkbzjfupucjcmq.supabase.co> – Supabase REST + Storage

4. API Endpoints Discovered in JS Bundle

Endpoint	Method (observed)	Purpose
/api/summary	GET	Business financial summary
/api/rows-by-status	GET	Policy rows by status
/api/agent-po-rows	GET	Agent PO/payment rows
/api/search-policy	GET	Policy search by number
/api/update-status	POST	Update row status
/api/bulk-update-status	POST	Bulk status update
/api/bulk-match-excel	POST	Excel matching
/api/update-agent-po	POST	Update agent PO
/api/batch-update	POST	Batch row update
/api/users	GET	User list
/api/data	GET	Full business data
/api/health	GET	Health check
/api/save-policy	POST	Save policy (localhost:3001)
/api/claude-extract	POST	AI extraction (localhost:3001)
/rest/v1/child_ids	GET	Supabase child IDs table
/rest/v1/policy_documents	GET	Supabase policy documents
/storage/v1/object/policy-documents/	GET	Supabase file storage

Verified Access

1. Unauthenticated API Data Leak – CONFIRMED

All six read-oriented API endpoints returned 200 OK with full data and **no authentication required**:

Endpoint	Status	Data Leaked
----------	--------	-------------

Endpoint	Status	Data Leaked
GET /api/summary	200	Financial totals: ₹418K business, ₹55K receivable, ₹277K payable, policy counts, premium breakdowns
GET /api/rows-by-status?status=all	200	16 rows of policy data including customer names, policy numbers, registration numbers, premium amounts, agent codes, broker/insurer names, payment details
GET /api/agent-po-rows	200	Agent PO data: policy numbers, payable amounts, payment references, broker/insurer details
GET /api/users	200	All 10 user accounts with usernames, passwords, roles, and emails
GET /api/data	200	165 rows of detailed business/policy data (Google Charts format)
GET /api/health	200	Server status + cached row count (165)

2. Supabase `child_ids` Table – Publicly Accessible

Using the exposed anon key, the Supabase `/rest/v1/child_ids` table returned **54 records** without any additional authentication. Each record contains:

- `agent_code`, `child_id` – agent identifiers
- `password_plain` – **plaintext passwords** for insurer portal logins (49 of 54 records have non-null, non-"string" passwords)
- `registered_email_id`, `registered_mobile_number` – PII
- `insurer_name`, `broker_name` – business relationships
- `security_question`, `security_answer` – security Q&A in plaintext
- `rm_name`, `rm_mobile`, `rm_email` – relationship manager PII

Sample plaintext passwords exposed: `Jio@9090`, `Sompo@123456`, `Bajaj@123456`, `Pace@123`, `Hdfc@007`, `Zuno@2025`, `uiic123`, `Office@2025`, `Simba@2025`, `Agent@123`, `Pass@1234`, `Sbi@123456`, `Password@123456`, `fI92$t26U`, and many more.

3. Supabase `policy_documents` Table – Exists but Restricted

The table exists (confirmed by 401 with `permission denied for table policy_documents` rather than 404), but the anon role lacks SELECT privileges. This is the only table that has any access control.

4. Write Endpoints – Not Triggered Without Auth

POST endpoints (`/api/update-status` , `/api/bulk-update-status` , `/api/bulk-match-excel` , `/api/update-agent-po` , `/api/batch-update`) returned `404` on GET, confirming they exist but only accept POST. Their POST access control was **not tested** (no destructive actions policy).

5. SPA Routes Accessible Without Auth

`/dashboard` , `/login` , `/signin` all returned `200` (the SPA shell). The login page renders the client-side authentication form that validates against the hardcoded user list.

Inferred Risks

CRITICAL – Full Database Exposure via Supabase

The Supabase `child_ids` table is completely open to the public internet with the anon key. Any attacker can read all 54 records containing plaintext passwords for insurer portals, PII (emails, phone numbers), and security Q&A. The anon key cannot be rotated without a redeployment, and it is valid until 2036.

CRITICAL – Unauthenticated Business Data Leak

Six API endpoints expose the entire business dataset (165+ policy records, financial summaries, agent payment data, customer PII) with zero authentication. This appears to be a backend that was developed without any auth middleware.

HIGH – Client-Side Only Authentication

Authentication is performed entirely in the browser against a hardcoded user list. The backend API does not enforce any server-side authentication or session management. The login form is a UI gate only – all data endpoints are directly accessible.

HIGH – Plaintext Password Storage

Passwords for insurer portal logins are stored in plaintext in the `password_plain` column of the Supabase `child_ids` table. This violates basic security practices and regulatory expectations for an insurance business.

MEDIUM – Hardcoded Credentials in Client Bundle

Ten user accounts with roles are shipped in the client-side JS. While 9 have placeholder passwords, the `gc1119 / GS@2024` account has a real password. The entire user list is extractable by anyone.

MEDIUM – Internal Service Reference

The JS bundle references `http://localhost:3001` for AI-powered policy extraction (`/api/claude-extract`) and policy saving (`/api/save-policy`). While not directly accessible externally, this reveals internal architecture and suggests a separate backend service exists.

LOW – SSL Certificate Issue

The site's SSL certificate fails standard verification (`CERTIFICATE_VERIFY_FAILED: unable to get local issuer certificate`), suggesting a misconfigured or non-public certificate chain.

Recommendations

1. **Immediately enforce authentication** on all `/api/*` endpoints. Implement server-side session or JWT validation – the current client-side-only auth provides zero protection.
 2. **Rotate the Supabase anon key** and redeploy. The current key is compromised in the client bundle and valid until 2036. Use Supabase Row Level Security (RLS) to restrict table access.
 3. **Enable RLS on the `child_ids` table** and all other tables. The `policy_documents` table already has some restrictions – apply the same to `child_ids`.
 4. **Remove plaintext passwords** from the database. Use a proper secrets manager or at minimum encrypt the `password_plain` column.
 5. **Remove hardcoded user accounts** from the client-side bundle. Move all authentication server-side with proper password hashing (bcrypt/argon2).
 6. **Fix the SSL certificate** chain so it passes standard verification.
 7. **Remove the localhost:3001 references** from the production bundle or ensure they are properly guarded.
 8. **Audit all Supabase storage buckets** and ensure no other tables or buckets are publicly accessible.
-

Limitations & Next Steps

- **POST endpoints not tested for auth:** The write/update endpoints (`/api/update-status`, `/api/bulk-update-status`, etc.) were only probed with GET (returned 404). Their POST

authentication was not tested per the no-destructive-actions policy. A follow-up authenticated test is recommended.

- **Supabase Storage:** The `policy-documents` bucket was not found, but other buckets may exist. A full bucket enumeration was not performed.
- **Supabase Realtime/WebSocket:** Not tested.
- **Other Supabase tables:** Only a subset of plausible table names were tested. The full schema was not enumerated.
- **The `policy_documents` table** requires elevated privileges — testing with a service role key was not performed (not available from the scan).